# CS683 Programming Assignment 2 Report

Bibaswan (24B0915), Aakash (24B1241), Sayan (24B0964), Joshua (24B1239)
4-Muxeteers

October 2026

## Contents

**1 Task 1** **1**
1.1 Performance Tables . . . 1
1.2 Performance Plots . . . 3
1.3 Storage Overhead Analysis . . . 3
1.3.1 IP Tracker Table . . . 3
1.3.2 Pattern Table . . . 4
1.3.3 Accuracy Tracker (Recent Prefetches) . . . 4
1.4 Throttling and Degree Adaptation . . . 4
1.4.1 Accuracy Throttling Thresholds . . . 4
1.4.2 Selection of Prefetch Degrees . . . 5

**2 Task 2** **5**
2.1 A. Instruction-Conscious L2 Replacement Policy (I-RRIP) . . . 5
2.2 B. Dynamic I-RRIP . . . 8

**3 Task 3: Incorporating SPP with Dynamic I-RRIP** **11**

---

**Note :** All raw ChampSim simulation logs are present in the `config_reports/` directory of our submitted repository.

---

# 1 Task 1

## 1.1 Performance Tables

Table 1: Baseline Performance and Prefetcher Metrics by Trace

| Trace | IPC | Speedup | L1D Load MPKI | Accuracy (%) | Coverage |
|---|---|---|---|---|---|
| Trace 1 | 0.2323 | 1 | 10.6780 | 91.1975 | - |
| Trace 2 | 0.2511 | 1 | 13.5379 | 96.8224 | - |
| Trace 3 | 0.3820 | 1 | 3.4034 | 96.6781 | - |
| **Average** | **0.2885** | **1** | **9.2064** | **94.8993** | **-** |

Table 2: IP-Stride Prefetcher Performance and Prefetcher Metrics by Trace

| Trace | IPC | Speedup | L1D Load MPKI | Accuracy (%) | Coverage |
|---|---|---|---|---|---|
| Trace 1 | 0.2334 | 1.0047 | 10.1190 | 91.1975 | 0.0543 |
| Trace 2 | 0.2512 | 1.0004 | 13.4378 | 96.8224 | 0.0078 |
| Trace 3 | 0.3828 | 1.0021 | 3.3181 | 96.6781 | 0.0253 |
| **Average** | **0.2891** | **1.0024** | **8.9583** | **94.8993** | **0.0291** |

Table 3: SPP Performance and Prefetcher Metrics by Trace

| Trace | IPC | Speedup | L1D Load MPKI | Accuracy (%) | Coverage |
|---|---|---|---|---|---|
| Trace 1 | 0.2345 | 1.0095 | 9.3951 | 91.1975 | 0.1259 |
| Trace 2 | 0.2548 | 1.0147 | 12.8810 | 96.8224 | 0.0518 |
| Trace 3 | 0.3919 | 1.0259 | 2.6698 | 96.6781 | 0.2170 |
| **Average** | **0.2937** | **1.0167** | **8.3153** | **94.8993** | **0.1316** |

## 1.2 Performance Plots

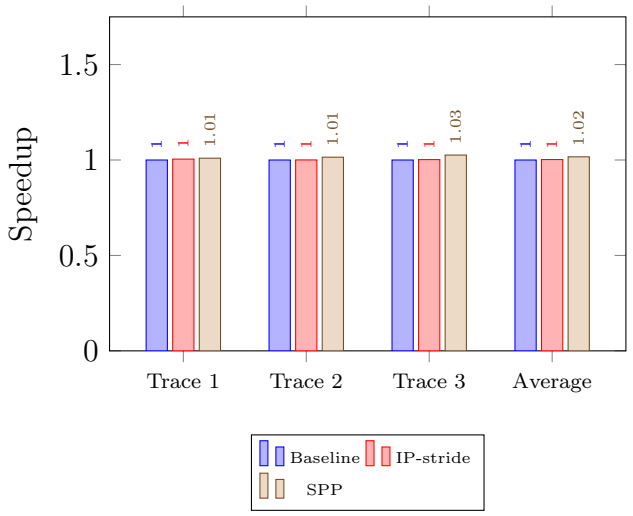



Figure 1: Speedups across traces

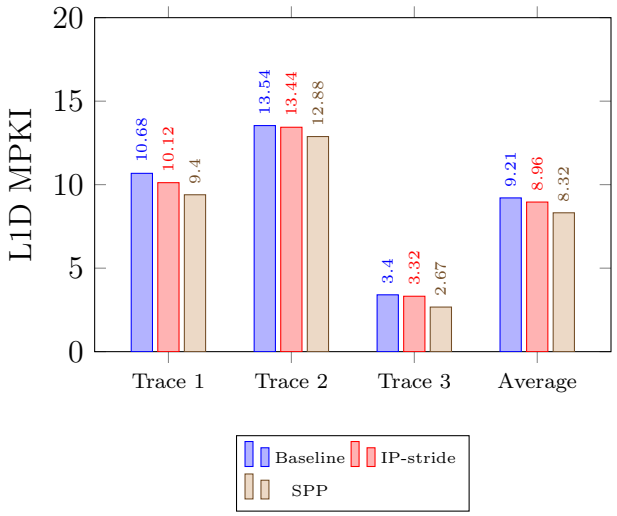



Figure 2: L1D MPKI across traces

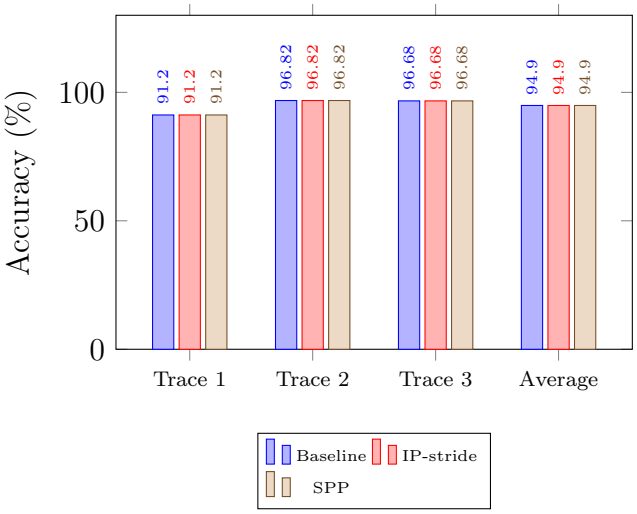



Figure 3: Accuracy across traces

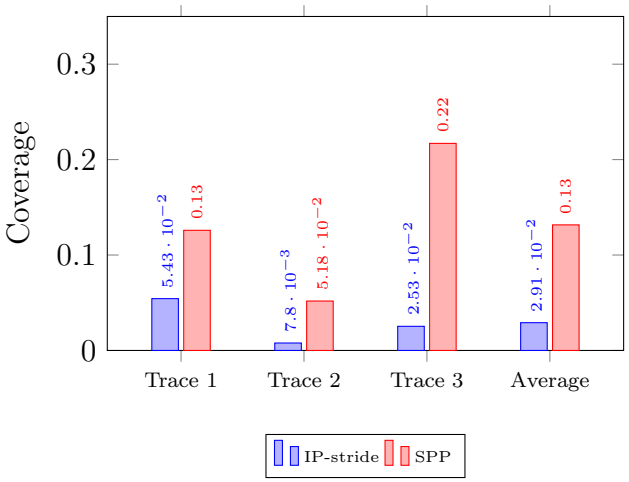



Figure 4: Coverage across traces

## 1.3 Storage Overhead Analysis

The total hardware storage overhead of the prefetcher is **124.1** KB. This is calculated based on the unoptimized data types used in the simulation code. A realistic hardware implementation could further compress these fields (e.g storing partial tags or smaller stride bounds).

### 1.3.1 IP Tracker Table

The IP Tracker is configured with 256 sets and 4 ways (1024 total entries).

- `tag`: 64 bits
- `last_block_addr`: 64 bits
- `last_stride`: 64 bits
- `last_last_stride`: 64 bits
- `confidence`: 32 bits (can be optimized to 2 bits in hardware)
- `last_used` (LRU): 64 bits (can be optimized to 2-3 bits for tree-LRU)
- `valid`: 1 bit
- **Total per entry:** 353 bits
- **Total Table Size:** $1024 \times 353 = 361,472$ bits ($\sim 44.1$ KB)

#### 1.3.2 Pattern Table

The Pattern Table contains 4096 direct-mapped entries.

- `tag` (14-bit signature padded to uint32): 32 bits *(14 bits strictly necessary)*
- `next_stride`: 64 bits *(typically optimized to 8-16 bits in hardware)*
- **Total per entry:** 96 bits
- **Total Table Size:** $4096 \times 96 = 393,216$ bits ($\sim 48.0$ KB)

#### 1.3.3 Accuracy Tracker (Recent Prefetches)

A flat array of 4096 entries to track recently issued prefetches.

- `recent_prefetches` element: 64 bits
- **Total Table Size:** $4096 \times 64 = 262,144$ bits ($\sim 32.0$ KB)

**Total Hardware Overhead:** $361,472 + 393,216 + 262,144 = 1,016,832$ bits = **124.1 KB**.

### 1.4 Throttling and Degree Adaptation

#### 1.4.1 Accuracy Throttling Thresholds

The prefetcher dynamically evaluates accuracy every 512 L1D loads. The throttling thresholds were set as follows:

- **High Accuracy Threshold (`ACCURACY_HIGH`):** Set to 50.0%.
- **Low Accuracy Threshold (`ACCURACY_LOW`):** Set to 25.0%.

**Decision Rationale:** We decided on an evaluation window of 512 L1D loads because it struck the best balance during our testing. If we evaluated more frequently, the prefetcher became too jittery and overreacted to tiny, random bursts of memory accesses. If we waited too long, it was too sluggish to adapt when the workload shifted to a new phase.

For the accuracy thresholds, we set the bar to increase aggressiveness at 50%. The logic here is fairly conservative: if we aren't hitting at least 50% chance of being right, pushing even more data into the small L1 cache will likely just waste memory bandwidth and evict useful data the CPU actually needs. On the other side, the 25% lower threshold acts as an emergency brake. Once accuracy drops that low, the prefetcher realizes it has lost the pattern and quickly throttles back to prevent severe cache pollution until the workload stabilizes again.

#### 1.4.2 Selection of Prefetch Degrees

The prefetch degree scales dynamically between a `MIN_DEGREE` of 1 and a `MAX_DEGREE` of 5, starting at a default of 2.

**Decision Rationale:** We capped the maximum degree at 5 simply because the L1 cache is small and highly sensitive to pollution. In our observations, trying to look ahead further than 5 blocks usually backfired—we ended up kicking out useful data the CPU was actually about to use, which caused thrashing.

For the minimum, we intentionally set it to 1 rather than shutting the prefetcher off completely during unpredictable phases. By leaving it at 1, it can continuously probe the memory stream so it's ready to quickly set itself back up the moment a predictable pattern returns.

Finally, we set the default starting degree to 2 as a safe middle ground. It gives a nice, immediate performance bump right out of the gate without overwhelming memory bandwidth while the accuracy tracker is still warming up and gathering its initial data.

# 2 Task 2

## 2.1 A. Instruction-Conscious L2 Replacement Policy (I-RRIP)

In this part, we had to modify the normal RRIP replacement policy to prioritize the instruction lines over data lines in the L2 cache. Our goal in designing the I-RRIP policy was to tackle the frontend stalls commonly seen in server workloads. Because these applications have large code footprints that easily overwhelm the L1 instruction cache, the L2 cache needs to try to retain instructions more than usual. To achieve this, we focused our modifications on the insertion and eviction phases while leaving the promotion logic untouched.

For this, we did the following:

- **Insertion:** In standard RRIP, all new lines are inserted with an RRPV of 2. We changed this to insert instruction lines at an RRPV of 1, while keeping data lines at 2. This gives incoming instructions a longer cache lifespan, helping them survive against streaming data workloads and reducing the chance of frontend starvation.

- **Promotion:** For promotion, we kept the default behavior where any cache hit promotes the line to an RRPV of 0. If a data line is actively getting hits, it has high temporal locality, so penalizing it during promotion would hurt data-heavy workloads without actually helping instruction fetches much.

- **Eviction:** We evict from RRPV=3 as usual. However, we prioritize data line eviction over instruction lines, so that instructions can stay for a longer time in cache and may have a later hit. First, we search for a data line at RRPV=3. Only if there is none do we search for an instruction line at RRPV=3. If RRPV=3 is empty, we increase every line's RRPV by 1 and follow the same step until we get a line to be evicted. This helps in reducing stalls by lowering the L2 instruction load MPKI.

### Performance Evaluation and Results

The table below summarizes the Instruction Per Cycle (IPC), Speedup (normalized to the LRU baseline), and the Misses Per Kilo-Instruction (MPKI) metrics for L2 overall, L2 Instructions, and L2 Data across the three traces.

| Trace | Policy | IPC | Speedup | L2C Load MPKI | L2 Inst MPKI | L2 Data MPKI |
|---|---|---|---|---|---|---|
| **Trace 1** | LRU (Base) | 0.2320 | 1.000x | 17.746 | 10.401 | 7.345 |
| | RRIP | 0.2319 | 0.999x | 17.675 | 10.595 | 7.080 |
| | I-RRIP | 0.2320 | 1.000x | 17.575 | 10.050 | 7.525 |
| **Trace 2** | LRU (Base) | 0.2504 | 1.000x | 28.769 | 20.172 | 8.597 |
| | RRIP | 0.2512 | 1.003x | 26.909 | 18.911 | 7.997 |
| | I-RRIP | 0.2551 | 1.019x | 24.236 | 15.307 | 8.929 |
| **Trace 3** | LRU (Base) | 0.3819 | 1.000x | 3.953 | 1.119 | 2.833 |
| | RRIP | 0.3816 | 0.999x | 4.029 | 1.187 | 2.843 |
| | I-RRIP | 0.3817 | 1.000x | 3.944 | 1.070 | 2.874 |
| **Average** | LRU (Base) | 0.2881 | 1.000x | 16.822 | 10.564 | 6.258 |
| | RRIP | 0.2882 | 1.001x | 16.204 | 10.231 | 5.973 |
| | I-RRIP | 0.2896 | 1.006x | 15.252 | 8.809 | 6.443 |

Table 4: Performance metrics comparing LRU, RRIP, and I-RRIP.

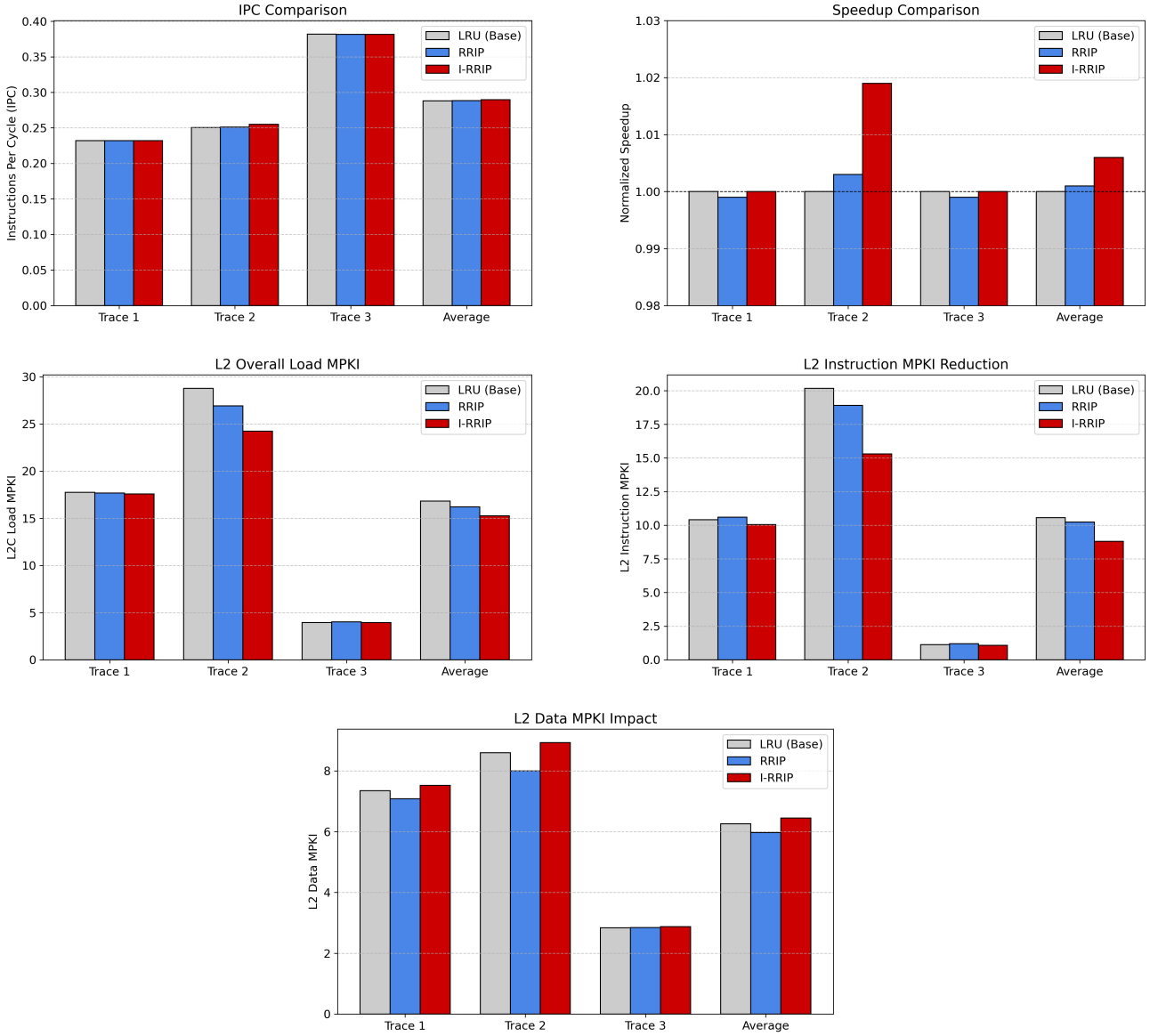



Figure 5: Visual comparison of IPC, Speedup, and MPKI metrics for Task 2A.

## 2.2 B. Dynamic I-RRIP

In this part, we extended our I-RRIP policy to dynamically switch between instruction prioritization and regular RRIP. Blindly prioritizing instructions all the time is not ideal because a program might go through phases where it is heavily fetching data. If we strictly prioritize instructions during these data-heavy phases, it will cause a huge increase in data misses and hurt the overall performance. Our goal was to use a Set Dueling approach so the cache can actually adapt to the workload's current behavior and only use I-RRIP when it is really needed.

For this, we did the following:

- **Leader Sets:** We needed a way to sample the cache behavior, so we chose 32 sets to act as our leader sets. Out of 2048 total L2 sets, 32 is a very small amount (about 1.5%). This keeps the hardware overhead very low and makes sure we don't mess up the overall cache capacity. We used a modulo interval `(set % (L2C_SET / 32) == 0)` to spread these leader sets evenly across the cache, which gives us a much more accurate sample of the memory access patterns. These leader sets always run the standard RRIP policy.

- **Tracking Counters:** To figure out what the workload is doing at any given time, we need to monitor the traffic going into the leader sets. We added counters for `total_access` and `total_miss`, and separate ones for `instr_access` and `instr_miss`. Whenever an access maps to one of our 32 leader sets, we update these counters. This gives us a real-time picture of whether the cache is currently bottlenecked by instructions or data.

- **Decision Logic:** For all the remaining normal sets (the follower sets), we use the counters to decide which policy to use. Before any insertion or eviction, we calculate the instruction intensity ratio: `(instr_access + instr_miss) / (total_access + total_miss)`. Based on our sensitivity testing, we picked a threshold of 0.40. If the ratio is strictly greater than 0.40, it means instructions are the main problem right now, so the follower sets switch to I-RRIP to protect them. If the ratio is lower, it means data accesses are dominating, so we fall back to normal RRIP. This ensures data lines are safe when the workload needs them.

### Threshold Sensitivity Study

To determine the optimal trigger point for dynamically switching to I-RRIP, we evaluated five different threshold values (0.10, 0.25, 0.40, 0.50, 0.75 and 0.90). A lower threshold aggressively favors instructions, while a higher threshold strictly reserves I-RRIP for extreme instruction-thrashing phases.

| Threshold | Average IPC | Avg Speedup | Avg L2 Load MPKI | Avg L2 Inst MPKI | Avg L2 Data MPKI |
|---|---|---|---|---|---|
| 0.10 | 0.2895 | 1.006x | 15.280 | 8.853 | 6.427 |
| 0.25 | 0.2896 | 1.006x | 15.273 | 8.844 | 6.429 |
| **0.40** | **0.2896** | **1.006x** | **15.272** | **8.844** | **6.429** |
| 0.50 | 0.2882 | 1.001x | 16.208 | 10.230 | 5.978 |
| 0.75 | 0.2882 | 1.001x | 16.204 | 10.231 | 5.973 |
| 0.90 | 0.2882 | 1.001x | 16.204 | 10.231 | 5.973 |

Table 5: Sensitivity analysis showing the trade-off between Instruction and Data MPKI.

As seen in the table setting the threshold to **0.40** yielded the optimal balance. Lower thresholds caused an unacceptable spike in L2 Data MPKI, while higher thresholds failed to trigger I-RRIP frequently enough to mitigate frontend stalls.

### Performance Evaluation and Results

The table below summarizes the Instruction Per Cycle (IPC), Speedup (normalized to the LRU baseline), and the Misses Per Kilo-Instruction (MPKI) metrics for L2 overall, L2 Instructions, and L2 Data across the three traces.

| Trace | Policy | IPC | Speedup | L2C Load MPKI | L2 Inst MPKI | L2 Data MPKI |
|---|---|---|---|---|---|---|
| **Trace 1** | LRU (Base) | 0.2320 | 1.000x | 17.746 | 10.401 | 7.345 |
| | RRIP | 0.2319 | 0.999x | 17.675 | 10.595 | 7.080 |
| | I-RRIP | 0.2320 | 1.000x | 17.575 | 10.050 | 7.525 |
| | **Dyn-IRRIP** | **0.2320** | **1.000x** | **17.570** | **10.058** | **7.512** |
| **Trace 2** | LRU (Base) | 0.2504 | 1.000x | 28.769 | 20.172 | 8.597 |
| | RRIP | 0.2512 | 1.003x | 26.909 | 18.911 | 7.997 |
| | I-RRIP | 0.2551 | 1.019x | 24.236 | 15.307 | 8.929 |
| | **Dyn-IRRIP** | **0.2550** | **1.019x** | **24.300** | **15.402** | **8.899** |
| **Trace 3** | LRU (Base) | 0.3819 | 1.000x | 3.953 | 1.119 | 2.833 |
| | RRIP | 0.3816 | 0.999x | 4.029 | 1.187 | 2.843 |
| | I-RRIP | 0.3817 | 1.000x | 3.944 | 1.070 | 2.874 |
| | **Dyn-IRRIP** | **0.3818** | **1.000x** | **3.949** | **1.073** | **2.875** |
| **Average** | LRU (Base) | 0.2881 | 1.000x | 16.822 | 10.564 | 6.258 |
| | RRIP | 0.2882 | 1.001x | 16.204 | 10.231 | 5.973 |
| | I-RRIP | 0.2896 | 1.006x | 15.252 | 8.809 | 6.443 |
| | **Dyn-IRRIP** | **0.2896** | **1.006x** | **15.273** | **8.844** | **6.429** |

Table 6: Final performance metrics comparing all L2 replacement policies.

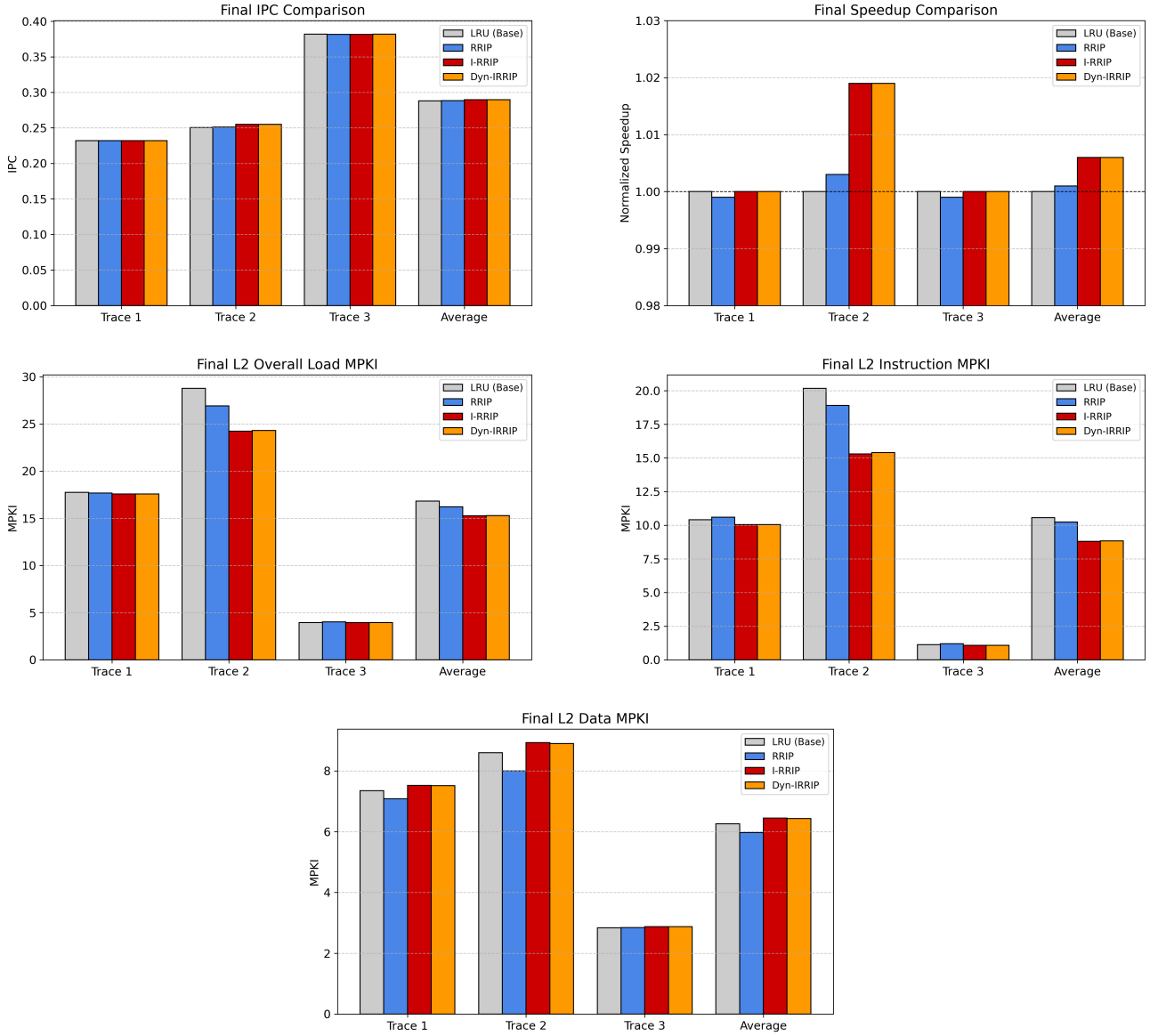



Figure 6: Final visual comparison of LRU, RRIP, I-RRIP, and Dyn-IRRIP metrics.

### Conclusion

While Dynamic I-RRIP successfully matches the 1.006x overall speedup of static I-RRIP, there is a negligible increase in the average L2C Load MPKI (from 15.252 to 15.273). This is because the 32 leader sets must continuously run the standard RRIP policy, incurring a small miss penalty during instruction-heavy phases to accurately monitor the workload.

# 3 Task 3: Incorporating SPP with Dynamic I-RRIP

When we combine an aggressive data prefetcher like SPP with our Dynamic I-RRIP, a big problem arises: Cache Pollution. The SPP prefetcher fetches a large amount of data blocks before they are actually needed. If we do not handle them carefully, these prefetched data blocks will fill up the L2 cache and kick out important instructions, ruining our frontend performance.

To fix this, we modified our replacement policy to be aware of prefetch packets. For this, we did the following:

- **Insertion:** When a block is brought in by a prefetch request, we insert it with an RRPV of 3 (maximum RRPV). This gives it the lowest possible priority. If the CPU actually uses this prefetched block later, it will get a cache hit and its RRPV will drop to 0. But if it is a useless prefetch, it will get evicted very quickly before it can do any harm to our instructions.
- **Eviction:** When looking for a victim block to evict, we first search for blocks that were prefetched but never actually used. We evict these unused prefetch blocks first. Only if there are none do we look for normal data blocks, and lastly, we look for instruction blocks. This acts as a strong shield for our instructions against prefetcher thrashing.
- **Counter Fix:** We also stopped counting prefetch requests in our Set Dueling leader sets. Because SPP generates a huge amount of data requests, it was skewing our instruction intensity ratio and tricking the dynamic policy into turning off I-RRIP during critical phases. By ignoring prefetch packets in the leader sets, our policy accurately measures the real demand of the CPU workload.

### Performance Evaluation and Results

The table below compares the naive implementation (SPP combined with standard Dynamic I-RRIP) against our Prefetch-Aware Dynamic I-RRIP.

As seen in the results, simply running a data prefetcher blindly alongside an instruction-conscious replacement policy actually hurts the L2 Instruction MPKI. In Trace 2, the naive integration caused the L2 Instruction MPKI to jump up to 17.374. By making the replacement policy explicitly aware of prefetch lines, we successfully pushed the L2 Instruction MPKI back down to 15.342, significantly dropping the overall L2C Misses and boosting our final average IPC to 0.2951. This outperforms both the standalone SPP and the standalone Dynamic I-RRIP policies.

| Trace | Policy | IPC | Speedup | L1D Acc. | L1D Cov. | L1D MPKI | L2C MPKI | L2 Inst MPKI | L2 Data MPKI |
|---|---|---|---|---|---|---|---|---|---|
| **Trace 1** | Baseline (LRU, No Pref) | 0.2321 | 1.000x | - | - | 10.679 | 17.746 | 10.401 | 7.345 |
| | Standalone Dyn-IRRIP | 0.2320 | 1.000x | - | - | 10.678 | 17.570 | 10.058 | 7.512 |
| | Standalone SPP | 0.2342 | 1.009x | 52.17% | 18.82% | 9.388 | 16.690 | 10.458 | 6.232 |
| | Naive Integration | 0.2342 | 1.009x | 52.38% | 18.81% | 9.390 | 16.678 | 10.659 | 6.020 |
| | **Prefetch-Aware (Final)** | **0.2343** | **1.009x** | **52.38%** | **18.76%** | **9.396** | **16.439** | **10.076** | **6.363** |
| **Trace 2** | Baseline (LRU, No Pref) | 0.2504 | 1.000x | - | - | 13.534 | 28.769 | 20.172 | 8.597 |
| | Standalone Dyn-IRRIP | 0.2550 | 1.018x | - | - | 13.538 | 24.300 | 15.402 | 8.899 |
| | Standalone SPP | 0.2539 | 1.014x | 45.39% | 7.32% | 12.875 | 28.489 | 20.542 | 7.947 |
| | Naive Integration | 0.2569 | 1.026x | 45.24% | 7.28% | 12.881 | 25.276 | 17.374 | 7.901 |
| | **Prefetch-Aware (Final)** | **0.2591** | **1.035x** | **45.11%** | **7.27%** | **12.882** | **23.546** | **15.342** | **8.204** |
| **Trace 3** | Baseline (LRU, No Pref) | 0.3819 | 1.000x | - | - | 3.404 | 3.953 | 1.119 | 2.833 |
| | Standalone Dyn-IRRIP | 0.3818 | 1.000x | - | - | 3.403 | 3.949 | 1.073 | 2.875 |
| | Standalone SPP | 0.3918 | 1.026x | 38.33% | 24.55% | 2.675 | 3.037 | 1.157 | 1.880 |
| | Naive Integration | 0.3915 | 1.025x | 38.39% | 24.55% | 2.675 | 3.099 | 1.213 | 1.886 |
| | **Prefetch-Aware (Final)** | **0.3918** | **1.026x** | **38.42%** | **24.69%** | **2.669** | **2.965** | **1.070** | **1.895** |
| **Average** | Baseline (LRU, No Pref) | 0.2881 | 1.000x | - | - | 9.206 | 16.823 | 10.564 | 6.258 |
| | Standalone Dyn-IRRIP | 0.2896 | 1.005x | - | - | 9.206 | 15.273 | 8.844 | 6.429 |
| | Standalone SPP | 0.2933 | 1.016x | 45.30% | 16.90% | 8.313 | 16.072 | 10.719 | 5.353 |
| | Naive Integration | 0.2942 | 1.020x | 45.34% | 16.88% | 8.315 | 15.018 | 9.749 | 5.269 |
| | **Prefetch-Aware (Final)** | **0.2951** | **1.023x** | **45.30%** | **16.91%** | **8.316** | **14.317** | **8.829** | **5.487** |

Table 7: Performance comparison demonstrating that Prefetch-Aware Dynamic I-RRIP successfully controls pollution and outperforms standalone implementations.

## Task 3 Performance Plots

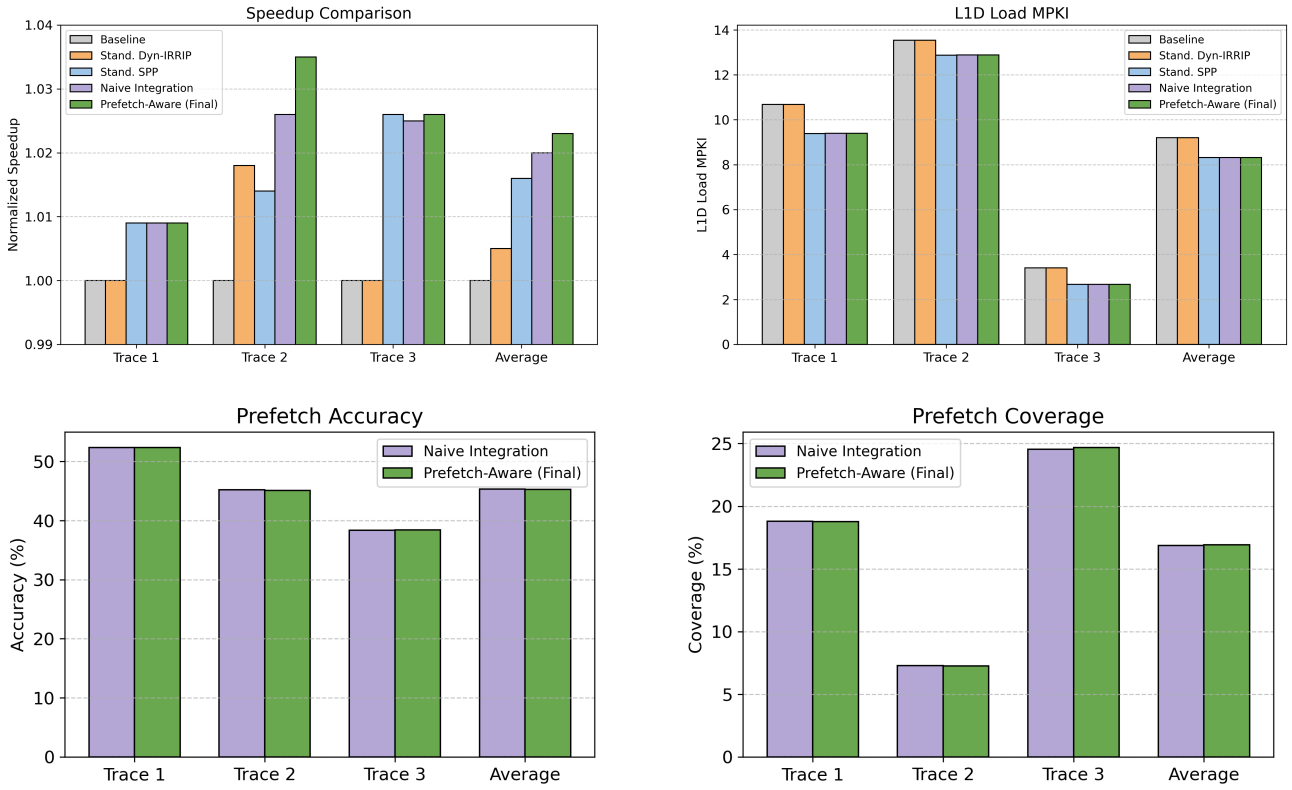



Figure 7: Speedup, L1D, and Prefetch metrics.

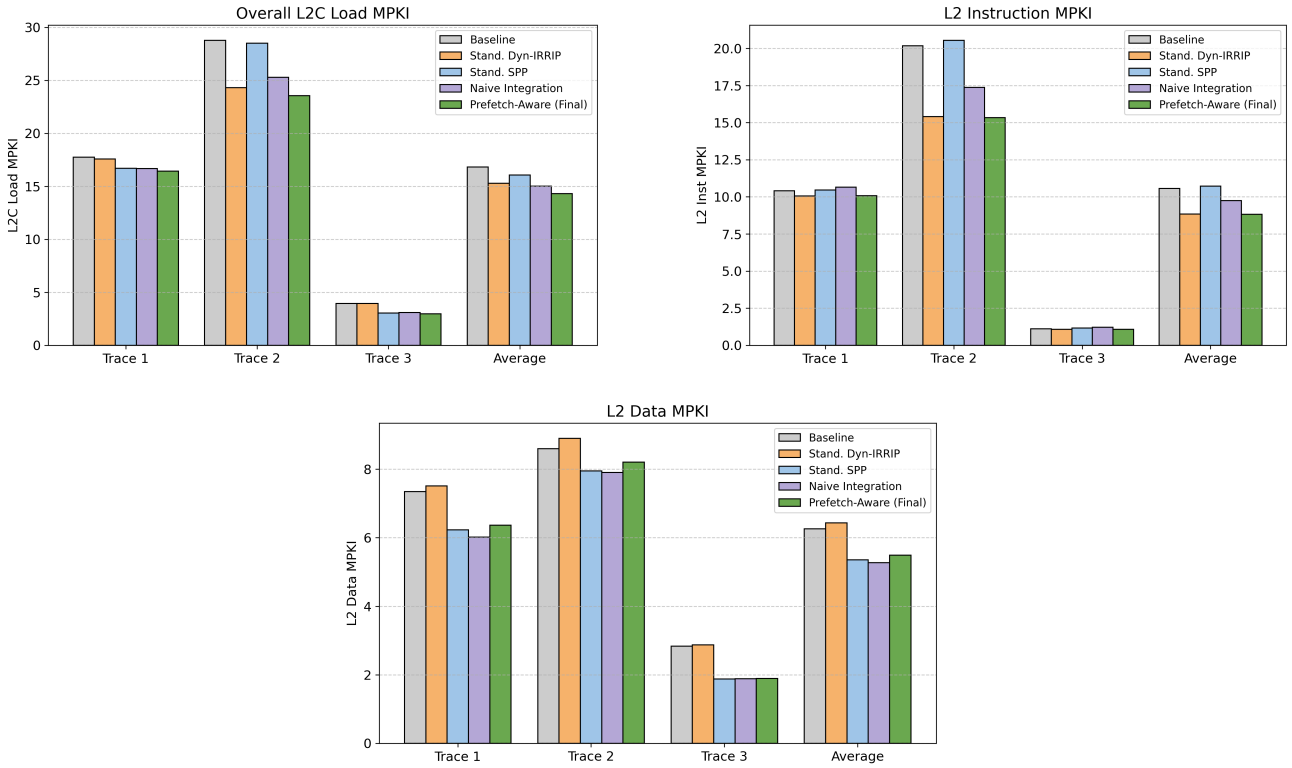



Figure 8: L2 Miss metrics comparing pollution control.